T1 FPPD

Problemas Clássicos de Concorrência

Gabriel Colling e Gabriel Postay

link do repositório no Github

: https://github.com/Gabriel-Colling/T1-FPPD

Problema 1: O Jantar dos Filósofos

1. Descrição da solução e decisões de projeto

O problema foi implementado com N goroutines (filósofos) e N channels de capacidade 1, cada um contendo um token que representa um garfo disponível. Cada filósofo alterna entre pensar, adquirir os dois garfos, comer e devolver, repetindo o ciclo R vezes. N e R são recebidos por parâmetros.

Na versão base, uma barreira faz todos pegarem primeiro o garfo esquerdo, provocando deadlock. Um channel de cancelamento e select permitem encerrar a demonstração. Na hierarquia, os garfos são adquiridos por ordem crescente de índice. Na limitação N-1, um channel de permissões impede que todos disputem os garfos ao mesmo tempo. sync.WaitGroup aguarda as goroutines, e time registra as esperas.

2. Análise de deadlock e starvation

O deadlock exige as quatro condições de Coffman: **exclusão mútua, hold and wait, ausência de preempção e espera circular**. Na base, todas se verificam: cada filósofo segura o garfo esquerdo e espera pelo direito. Na hierarquia, a ordem global de aquisição quebra a espera circular. No limite N-1, ao menos um filósofo fica fora da disputa, impedindo o bloqueio circular completo. As duas soluções evitam deadlock, mas não garantem starvation-free, pois não há política estritamente justa de acesso aos recursos.

Versão	Deadlock	Starvation
Base	Ocorre (4 condições de Coffman)	Não há progresso global; deadlock
Hierarquia	Não ocorre (quebra da espera circular)	Possível
Limite N-1	Não ocorre (limitação de acesso)	Possível

3. Análise de fairness (N = 5, R = 10)

A tabela apresenta as refeições concluídas e o tempo médio de espera (ms) por filósofo. Na hierarquia, o tempo inclui a aquisição dos garfos, no limite N-1 inclui também a espera pela permissão de entrada. A média é calculada dividindo a soma dos tempos de espera pelo número de refeições concluídas.

Filósofo	Base (refeições)	Hierarquia (ref.; ms)	Limite N-1 (ref.; ms)
0	0	10; 23,248	10; 468.472
1	0	10; 134.248	10; 401.637
2	0	10; 67.527	10; 668.998
3	0	10; 0.384	10; 602.244
4	0	10; 401.763	10; 535.282
Total	0	50; —	50; —

Na base, nenhuma refeição foi concluída, impossibilitando calcular média por refeição. Hierarquia e limite N-1 concluíram 50 refeições cada. As médias variaram 125,434 ms na hierarquia e 535,326 ms no limite N-1, indicando distribuição mais uniforme na primeira estratégia nesta execução. A igualdade de refeições é esperada, pois cada filósofo deve completar R ciclos, não comprova fairness. Os tempos dependem do escalonamento e não eliminam o risco de starvation.

Encerramento ordenado — Hierarquia de recursos: Nessa estratégia, cada filósofo executa R iterações, adquirindo os garfos em ordem crescente de índice. Após cada refeição, os dois garfos são devolvidos aos respectivos channels, permitindo que outros filósofos os utilizem. Ao completar todas as refeições, a goroutine do filósofo é encerrada. O `sync.WaitGroup` garante que a função principal aguarde a finalização de todos os filósofos antes de encerrar o programa, evitando goroutines pendentes.

Encerramento ordenado — Limitação N-1: Cada filósofo solicita uma permissão de entrada por meio de um channel com capacidade N-1. Após adquirir os garfos e realizar a refeição, devolve ambos os recursos e libera sua permissão, permitindo a entrada de outro filósofo. Esse processo se repete até completar R refeições. Ao finalizar, cada goroutine sinaliza sua conclusão pelo

`sync.WaitGroup`, garantindo que o programa termine somente quando todos os filósofos tiverem encerrado suas atividades e liberado os recursos compartilhados.

Problema 2: Produtor/Consumidor

1. Solução e decisões.

Channels: O buffer é um canal do Go. O bloqueio (cheio/vazio) e a sincronização são gerenciados nativamente pelo *scheduler* da linguagem.

Semáforos (Memória Compartilhada): Utilizamos um array circular. A principal decisão foi o Locking Granular: mutexes separados para leitura e escrita, permitindo que um produtor e um consumidor acessem o buffer paralelamente. Canais vazios de tamanho K simularam semáforos de contagem (notEmpty e notFull). Em ambas, usamos sync.WaitGroup para orquestrar as *goroutines* e sync/atomic para prevenir *Data Races* nos contadores.

2. Análise de Deadlock e Starvation

As soluções são livres de ambos:

Deadlock: Na versão de semáforos, evitamos as Condições de Coffman impondo uma ordem estrita de aquisição: o produtor garante espaço (Wait(notFull)) antes de travar o índice (Mutex.Lock). Isso quebra a "Espera Circular" e o *Hold and Wait*.

Starvation: Evitada graças à Justiça do escalonador do Go. Goroutines bloqueadas entram numa fila FIFO. Os dados comprovaram consumo balanceado ($\sim$ 200 itens por consumidor), sem starvation de nenhuma rotina.

3. Efeito do Buffer e Comparação das Versões

K = 1: Alto acoplamento temporal. Produtores e consumidores precisam estar sincronizados. O *throughput* é o menor devido às constantes trocas de contexto.

K = 10 e K = 100: O buffer atua como amortecedor (*shock absorber*). O desacoplamento temporal aumenta exponencialmente e o *throughput* atinge o limite do hardware, pois variações momentâneas de velocidade não travam as rotinas vizinhas.

Comparação: A versão com *Channels* é mais limpa e possui otimizações de baixo nível do compilador. A versão com *Semáforos* exige mais código de gestão, mas comprova na prática a robustez do algoritmo clássico de Dijkstra para ambientes sem mensageria nativa.

4. Encerramento Ordenado

Versão Channels: A rotina main executa close(buffer). Os consumidores continuam lendo até o buffer esvaziar, reconhecendo o fim através do *comma-ok idiom* (ok == false), realizando um retorno limpo.

Versão Semáforos: Utilizamos select com time.After. Se o consumidor ociosamente aciona o *timeout*, ele verifica uma *flag* global protegida (done == true) e

o tamanho do semáforo notEmpty. Se a produção acabou e a fila está vazia, a rotina é encerrada, prevenindo Deadlock na linha de leitura.

Referências Utilizadas

- **SILBERSCHATZ, A. et al. (Operating System Concepts):** Utilizado para a estrutura do Buffer Circular clássico, orquestração de semáforos (Wait/Signal) e fundamentos para prevenção das Condições de Coffman (Deadlock).
- **DONOVAN, A.; KERNIGHAN, B. (The Go Programming Language):** Referência para a implementação do padrão CSP, funcionamento do *Go Scheduler*, *comma-ok idiom* para canais e o uso do construto select com *timeouts*.
- Producer-consumer in Go | IBM
- Producer-Consumer Pattern in Go - DEV Community
- PANTUZA, Gustavo. **O jantar dos filósofos: problema de sincronização em sistemas operacionais**. Blog de Ciência da Computação, 26 out. 2018. Disponível em: https://blog.pantuza.com/artigos/o-jantar-dos-filosofos-problema-de-sincronizacao-em-sistemas-operacionais.

Uso de IA Filósofos(ChatGPT)

Utilizei para organização do README do trabalho.

Validação da implementação do problema com sincronização e limite de filósofos.

Auxiliou nas implementações das métricas de avaliação

Utilizei para problemas enfrentados na hora de rodar go run -race.

Produtor/consumidor(GEMINI):

Foi utilizada IA para a implementação da coleta de dados das execuções com diferentes Ks de maneira que funcionasse com apenas uma execução.

Usada para discutir quais métodos eram mais eficientes e por que tentativas anteriores não satisfaziam as condições impostas.

Usada para me ajudar com o entendimento do fluxo e como as coisas deveriam funcionar no código para resolver o problema, visto que não consegui resolver apenas com leituras.

Usada para me ajudar com as referências e entender melhor os exemplos propostos que encontrei online e me recomendar soluções na literatura.